Lab 9.1: Using the Web Browser Control

In this lab, you will implement the **Web Browser ActiveX** control in an application.

To see a demonstration of the solution for this lab, click this icon.
(CD-ROM plays the demonstration, "Lab 9.1 Demonstration.")

Estimated time to complete this lab: **40 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch09\Lab9.1.

The solution code for this lab is located in the folder \Labs\Ch09\Lab9.1.

Objectives

After completing this lab, you will be able to:

- Create an MFC-based application that is a control container.
- Use the Developer Studio Gallery to include the **Web Browser** control in your project.
- Programmatically create a **Web Browser** control and set properties and invoke methods of the **Web Browser** control.
- Handle events of the **Web Browser** control in your application.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercises provide practice working with the concepts and techniques covered in this chapter.

Exercise 1: Using the Web Browser Control

In this exercise, you will create a simple control container application and use the Developer Studio Gallery to add the **Web Browser** control to the project. You also will add code to create an instance of the **Web Browser** control and to invoke a method and set properties of the control.

Exercise 2: Handling Web Browser Control Events

In this exercise, you will modify the application created in Exercise 1 of this lab to handle a Web Browser control event.

300 Exercise 1: Using the Web Browser Control

In this exercise, you will create a simple a control container application and use the Microsoft Developer Studio Gallery to add the **Web Browser** control to the project. You also will add code to create an instance of the **Web Browser** control and to invoke a method and set properties of the control.

▶ Create a control container application

☐ 1. Start Developer Studio and then on the **File** menu, click **New**.

☐ 2. In the **New** dialog box, click the **Projects** tab, and select **MFC AppWizard (EXE)**.

☐ 3. In the **Location** field, type the directory path under which you want to create the project. Type **Browse** in the **Project Name** field, accept the default option **Create new workspace**, then click **OK**.

☐ 4. In Step 1 of MFC AppWizard, select **Single Document** for the type of application to create, and then click **Next**.

☐ 5. In Step 2 of MFC AppWizard, accept the default settings and click **Next**.

☐ 6. In Step 3 of MFC AppWizard, accept the default of **None** for compound document support and under other support, make sure that the **ActiveX Controls** check box is checked. You can accept defaults for the remaining steps, so click **Finish**.

▶ Add the Web Browser control to your project

☐ 1. On the **Project** menu, click **Add To Project**, and then click **Components and Controls**. In the **Gallery** dialog box, double-click the **Registered ActiveX Controls** folder.

☐ 2. Select the **Microsoft Web Browser Control** component, click **Insert,** and then click **OK**.

☐ 3. In the **Confirm Classes** dialog box, click **OK**.

☐ 4. In the **Gallery** dialog box, click **Close**.

Note that the **CWebBrowser2** class has been added to the list of classes in the Browse application.

☐ 5. In the ClassView pane, expand the **CWebBrowser2** class to view properties and methods of the class.

▶ Create an instance of the Web Browser control

☐ 1. Before you create an instance of the **Web Browser** control, you will need to include the header file for the **CWebBrowser2** class in the header file of the view class.

☐ 2. You also need to add a member variable for a pointer to a **Web Browser** control to the class in which you will create the control. In this case, you will embed the control in the view class.

a. In the ClassView pane, right-click **CBrowseView**, then click **Add Member Variable**.

b. In the **Add Member Variable** dialog box, type **CWebBrowser2*** in the **Variable Type** field.

c. Type **m_pBrowse** in the **Variable Name** field.

d. Accept the default value of **Public access,** and click **OK**.

e. In BrowseView.cpp, add the following line to the constructor **CBrowseView::CBrowseView**:

```
m_pBrowse = NULL;
```

Notes

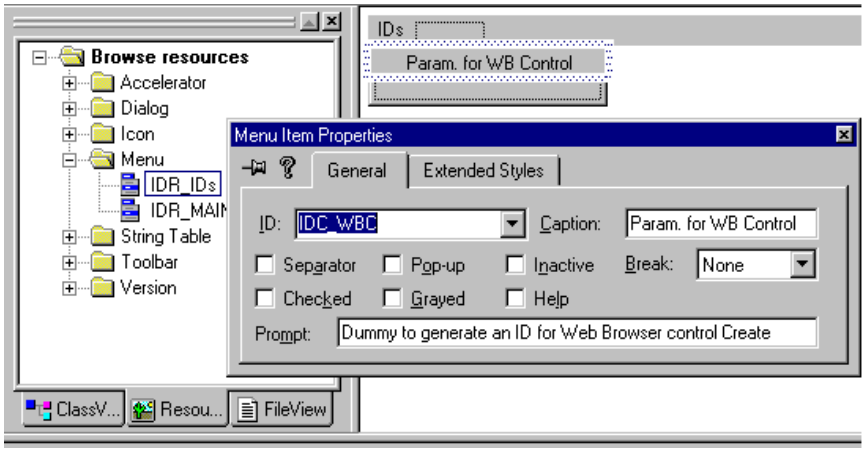
☐ 3. Create a control ID to use as a parameter for the **Web Browser** control **Create** function.

a. In the ResourceView pane, right-click the **Menu** folder and select **Insert menu**.

b. In the Menu editor pane, double-click the empty top-level menu item provided and in the **Menu Item Properties** dialog box, set the **Caption** property to **IDs**.

c. In the Menu editor pane, double-click the empty menu item under IDs. In the **Menu Item Properties** dialog box, set the **ID** property to **IDC_WBC**. This is the control ID you will provide as a parameter to the **Web Browser** control's **Create** function in the next step.

You can add caption and prompt information to help you remember what this resource is for.



☐ 4. To embed the control in the view, add a message handler for the WM_CREATE message to the view class, and place the code to create an instance of the **Web Browser** control in the message handler.

a. In the ClassView pane, right-click **CBrowseView**, and then click **Add Windows Message Handler**.

b. From the list of handlers, select **WM_CREATE**, then click **Add and Edit**.

c. In the **CBrowseView::OnCreate** function, add the following code before the return statement:

```
// Define the area where the control will reside.
CRect rect;
GetClientRect(&rect);

// Create the control.
// IDC_WBC is a unique identifier for the control and was defined
// in Step 3 using a dummy menu item.
m_pBrowse = new CWebBrowser2;
ASSERT(m_pBrowse);
if (!m_pBrowse->Create(NULL,NULL,WS_VISIBLE,rect,this,IDC_WBC))
{
    TRACE("failed to create browser\n");
    delete m_pBrowse;
    m_pBrowse = NULL;
    return 0;
}
```

Notes

You will not be able to see any results from the code you have added until you have completed the next two steps of invoking a method and setting properties of the control.

▶ Invoke a method of the Web Browser control

- Following the lines added to the **CBrowseView::OnCreate** function in the previous procedure, add the code shown below to invoke the **Web Browser** control's **Navigate** method:

```
// Initialize the first URL.
COleVariant noArg;
m_pBrowse->Navigate("www.microsoft.com", &noArg, &noArg, &noArg,
                    &noArg);
```

Replace the string "www.microsoft.com" with the Web site name created by the Personal Web Server.

Note The empty **COleVariant** arguments can be set to further customize the **Navigate** method. For more information about **COleVariant** arguments, search for "COleVariant::COleVariant" in MSDN Library Visual Studio 6.0.

▶ Set properties of the Web Browser control

To set properties of the control, add a message handler for the WM_SIZE message to the view class, and place the code to set the height and width properties of the **Web Browser** control in the message handler.

☐ 1. In the ClassView pane, right-click **CBrowseView**, and then click **Add Windows Message Handler**.

☐ 2. From the list of handlers, select **WM_SIZE**, then click **Add and Edit**.

☐ 3. In the **CBrowseView::OnSize** function, add the following code after the call to the base class's **OnSize** function:

```
if (m_pBrowse)
{
    m_pBrowse->SetWidth(cx);
    m_pBrowse->SetHeight(cy);
}
```

This code causes the **Web Browser** control to resize when a user resizes the application's mainframe window.

▶ Build and test your application

- If you have an active connection to the Internet, you should see the default page for the Microsoft Corporation's Web site displayed in the **Web Browser** control in your application's view class.

The solution code for this exercise is located in the folder \Labs\Ch09\Lab9.1\Ex01\Solution.

Notes

300 Exercise 2: Handling Web Browser Control Events

Continue with the files you created in Exercise 1 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch09\Lab9.1\Ex02.

In this exercise, you will modify the application from Exercise 1 of this lab to handle a **Web Browser** control event.

There are three main steps to handle events:

- Determine the function prototype and ID of the event.
- Add the event function handler to the **Wrapper** class.
- Link the event to the handler.

▶ Determine the function prototype and ID of the event

☐ 1. On the Developer Studio **Tools** menu, click **OLE/COM Object Viewer**.

☐ 2. In the left pane of the **OLE/COM Object Viewer** dialog box, double-click the **Controls** folder. Scroll down the list and right-click **Microsoft Web Browser Control**.

☐ 3. On the menu that appears, click **View Type Information**.

☐ 4. In the **ITypeLib Viewer** dialog box that appears, double-click **dispInterface DWebBrowserEvents2**, and then double-click the **Methods** folder.

This displays the event methods for the **Web Browser** control.

 (CD-ROM displays the following tip.)

If you do not see the dispInterfaceDWebBrowserEvents node in the left-hand pane of the ITypeLib Viewer dialog box, on the View menu, see whether "Group by type kind" is checked. If so, click Group by type kind to uncheck it and the dispInterfaceDWebBrowserEvents node will appear.

☐ 5. In the **Methods** folder, click **StatusTextChange**. The information you require about the ID for the event and the prototype for the event handler appears in the right pane of the **ITypeLib Viewer** dialog box:

```
[id(0x00000066)
void StatusTextChange([in] BSTR Text);
```

▶ Add the event function handler to the control's Wrapper class

☐ 1. Before you create the **OnStatusTextChange** event handler, create a public member function, **Status**, in the **CMainFrame** class. This helper function, which enables the view class to access the status bar object, will be called in the **OnStatusTextChange** event handler.

a. In the ClassView pane, right-click **CMainFrame**, and then click **Add Member Function**.

b. In the **Add Member Function** dialog box, type **void** in the **Function Type** field, type **Status(LPCTSTR text)** in the **Function Declaration** field, then click **OK**.

c. Add the following line to the **Status** function:

```
m_wndStatusBar.SetPaneText(0,text,TRUE);
```

Notes

☐ 2. You can now add the **OnStatusTextChange** event handler function. In the ClassView pane, right-click **CBrowseView**, then click **Add Member Function**.

☐ 3. In the **Add Member Function** dialog box, type **void** in the **Function Type f**ield, type **OnStatusTextChange(LPCTSTR text)** in the **Function Declaration** field, then click **OK**.

☐ 4. Add the following lines to the **OnStatusTextChange** function:

```
// Only write to status line if there is a non-zero string.
// (lstrlen works on either ANSI or UNICODE strings.)
if (lstrlen(text))
    ((CMainFrame*)AfxGetMainWnd())->Status(text);
```


(CD-ROM displays the following tip.)

AfxGetMainWnd returns a **CFrameWnd** pointer. You must cast the return value to a **CMainFrame** pointer because **Status** is a member function of the **CMainFrame** class.

☐ 5. Because the **Status** function called in the previous step is a member of the **CMainFrame** class, add the following line to BrowseView.cpp file to include the header file for the **CMainFrame** class:

```
#include "mainfrm.h"
```

▶ Link the event to the handler

In this application, the view class handles the **OnStatusTextChange** event, so the changes outlined below are made in each case to the appropriate view class file(s).

☐ 1. Add a DECLARE_EVENTSINK_MAP() macro call to the header file for the class that handles the event.

a. Open the BrowseView.h file.

b. Add the following line after the call to `DECLARE_MESSAGE_MAP()`:

```
DECLARE_EVENTSINK_MAP()
```

☐ 2. Add an event sink map to the source file for the class that handles the event. Recall that the event ID displayed in the OLE/COM Object Viewer for the **StatusTextChange** event was 0x66.

a. Open BrowseView.cpp.

b. After the line `END_MESSAGE_MAP()`, add the following lines to add an event sink map to the view class:

```
BEGIN_EVENTSINK_MAP(CBrowseView, CView)
    ON_EVENT(CBrowseView, IDC_WBC, 0x66, OnStatusTextChange, VTS_BSTR)
END_EVENTSINK_MAP()
```

▶ Build and test your application

- You will see that the text in the status bar changes as the **Web Browser** control navigation takes place.

The solution code for this exercise is located in the folder \Labs\Ch09\Lab9.1\Ex02\Solution.

Notes